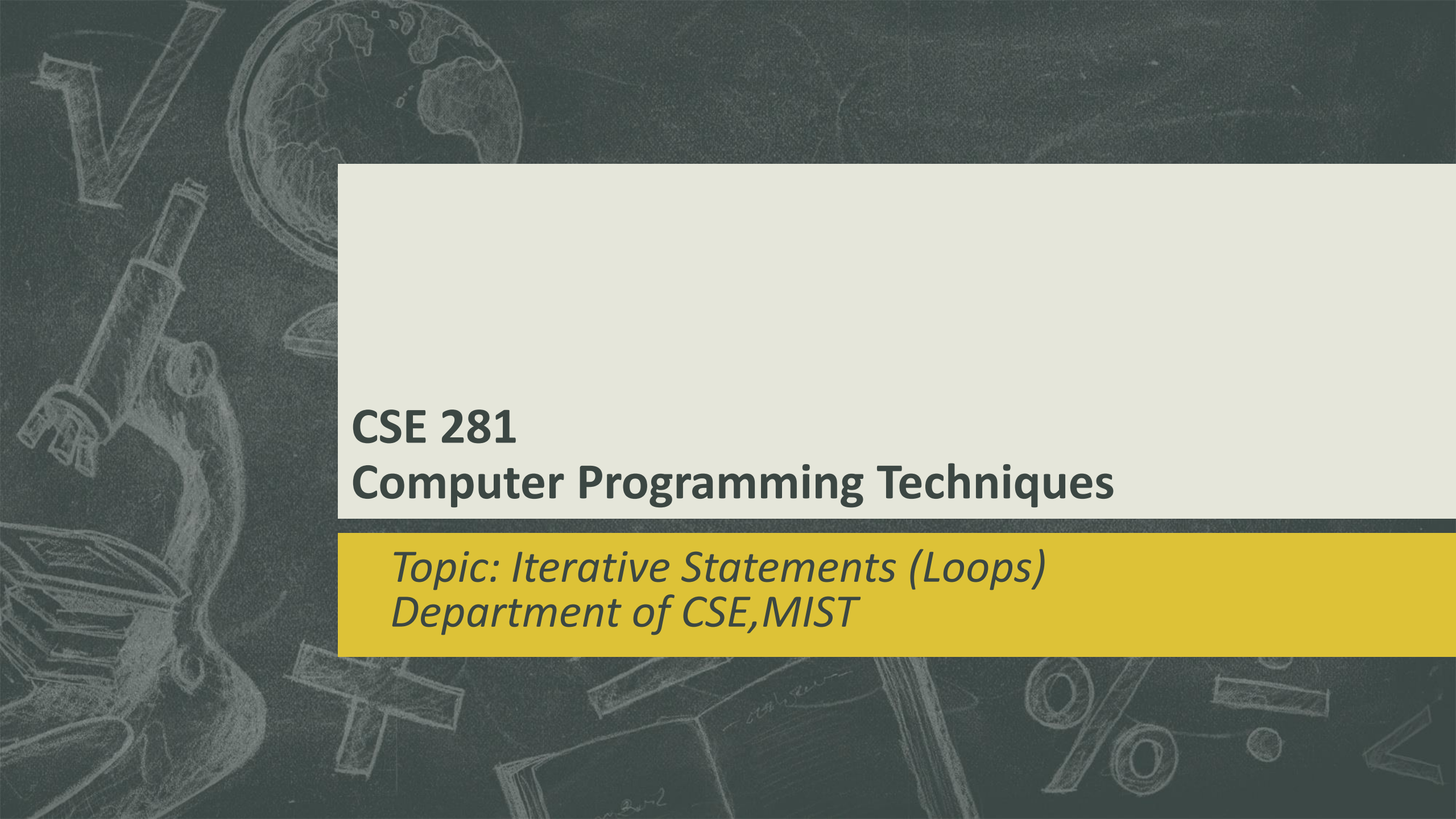




CSE 281

Computer Programming Techniques

Topic: Iterative Statements (Loops)
Department of CSE, MIST

Why are Loops used?

- Write a code to print hello world

```
#include <stdio.h>
int main() {
    printf( "Hello World\n");
}
```

Why Loop is used?

- **Write a code to print hello world 10 times**

```
#include <stdio.h>
```

```
int main() {
```

```
    printf( "Hello World\n");
```

```
    printf( "Hello World\n");
```

```
    printf( "Hello World\n");
```

```
    printf( "Hello World\n");
```

```
    printf( "Hello World\n");
```

```
    printf( "Hello World\n");
```

```
    printf( "Hello World\n");
```

```
    printf( "Hello World\n");
```

```
    printf( "Hello World\n");
```

```
    printf( "Hello World\n"); }
```


Why Loop is used?

- **Loops** in C programming is **used** when we need to **execute a block of statements repeatedly**.
- For example: Suppose we want to print “Hello World” 10 times. How can this be done efficiently?
- Using **loop** statement, we could easily solve this problem **without writing** the **printf** statement 10 times.

Solution 1:

```
#include <stdio.h>
```

[illegible]

Why Loop is used?

- In looping, a sequence of statements are executed until some conditions for termination of the loop are satisfied.
- In short, in loop statement, we can **specify** how many times a block of statement will be executed.
- There are three loop statements in C:
 - for loop
 - while loop
 - do...while loop

for Loop - syntax

- **For loop** is used to repeat a statement or a block of statements a specific number of times.

- **Syntax:**

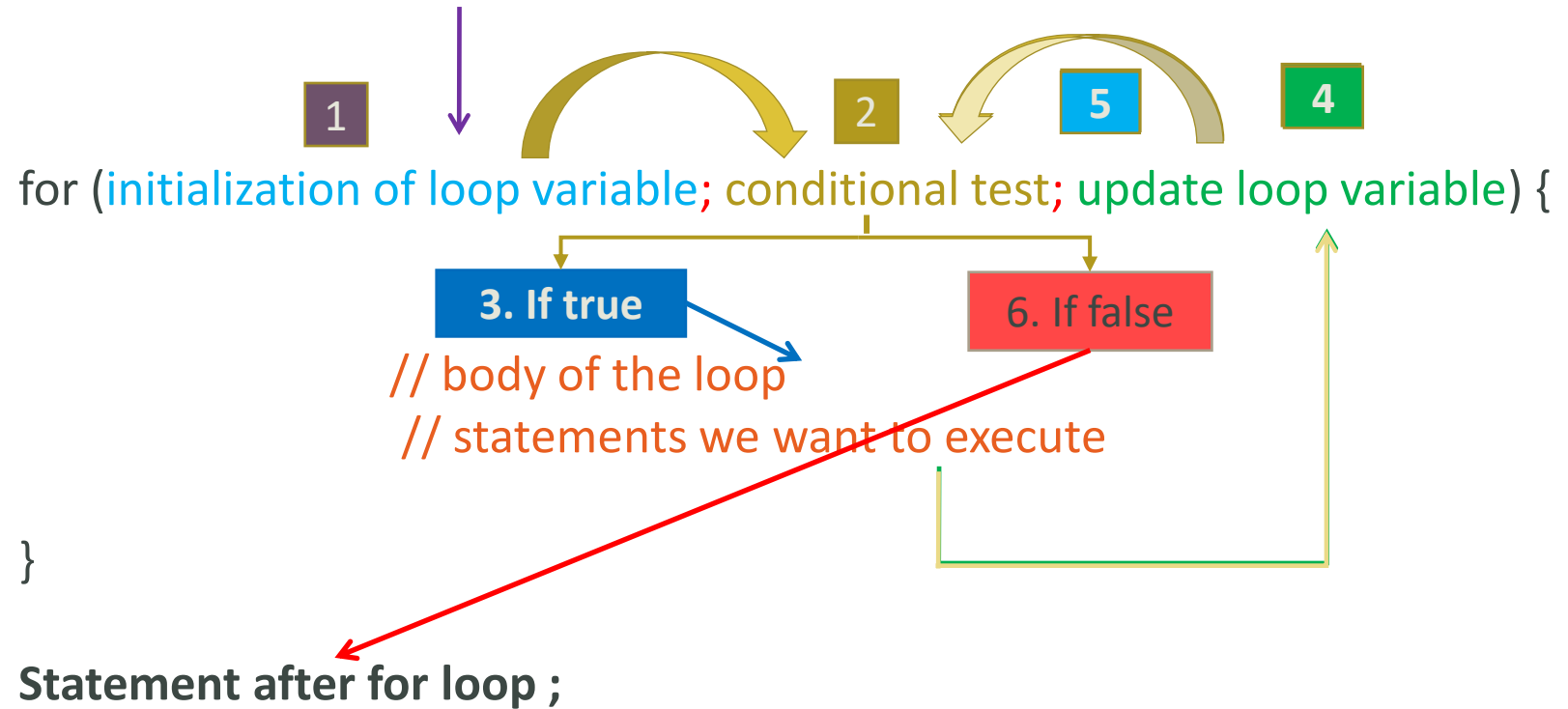
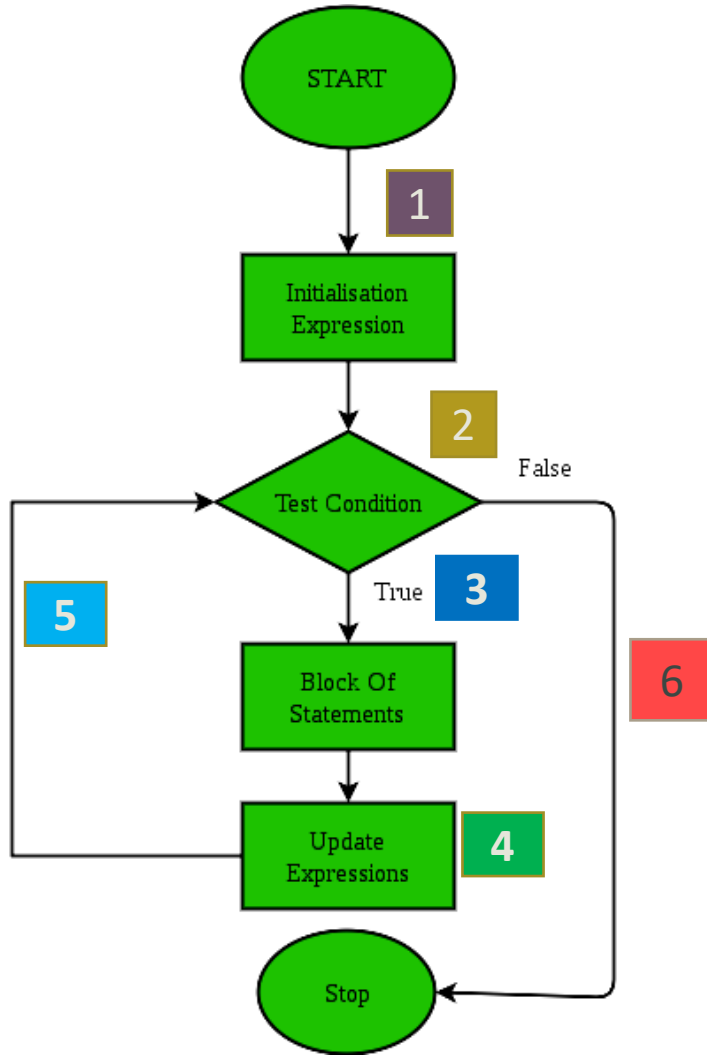
```
for (initialization of loop variable; conditional test; update loop variable) {  
    // body of the loop  
    // statements we want to execute  
}
```

for Loop – Execution Steps

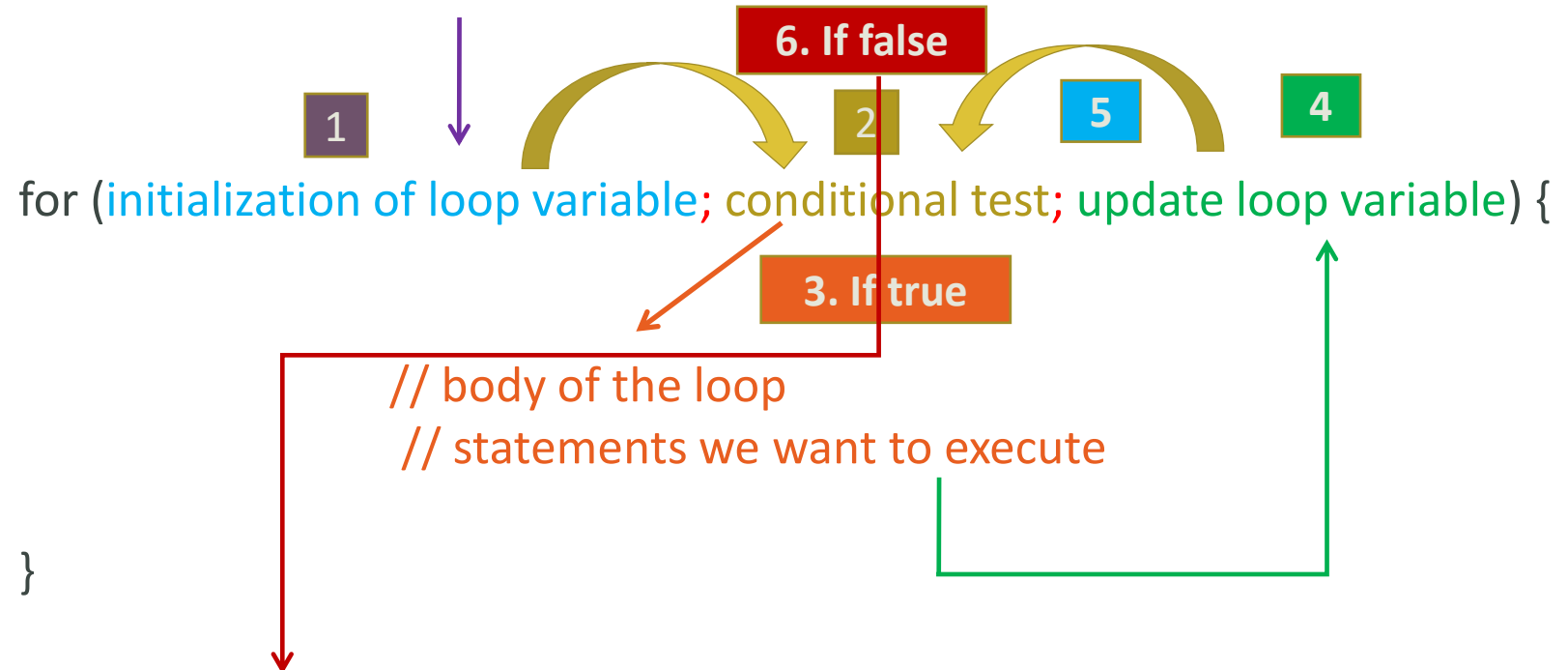
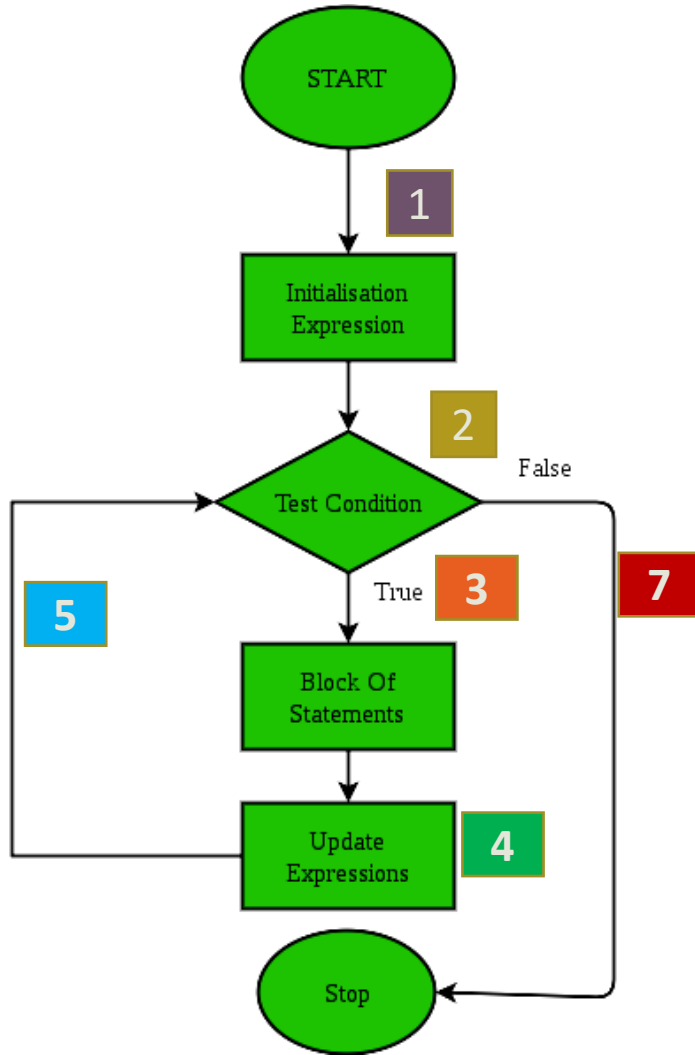
```
for (initialization of loop variable; conditional test; update loop variable) {  
    // body of the loop  
    // statements we want to execute  
}
```

1. At first the loop variable is **initialized**. This statement is executed **only once**.
2. Then **condition** is **evaluated**-
 - i. If the condition is **false**, it returns 0 and the **loop** is **terminated**.
 - ii. if the condition is **true**, it returns 1, **body of the loop is executed**, **loop variable is updated**
3. Step 2 is **repeated until** the loop is **terminated**

for Loop – Flow Chart



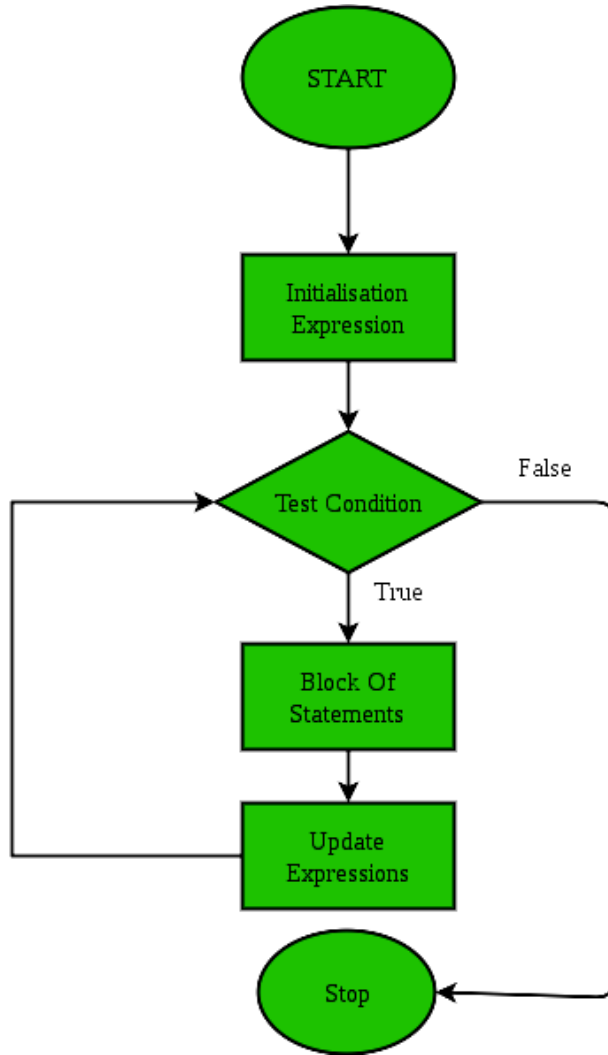
for Loop – Flow Chart



Statement after for loop ;

Terminate Loop and execute the statement right after the loop

Summary of for loop Execution



1. Initialize loop variable
2. Check condition
3. if **false terminate** loop body
4. If **true execute** loop body
5. Update loop variable and repeat from step 2

Sample for Loop

```
for (i=0 ; i<2 ; i++) {  
    printf("Hello World");  
}  
printf("Outside loop body");
```

OUTPUT

Hello World
Hello World
Outside loop body

1. Here at first loop variable **i** is **initialized** with **0**
2. As step , condition **i<2** is **evaluated**. Since now value of **i** is **0** which makes the condition **i<2** **true**, the body of the for loop will be executed.
3. Then the loop variable **i** will be updated. As we have **i++**, value of **i** now will be 1
4. Condition **i<2** is **evaluated** and holds **true** for **i=1**, the body of the for loop will be executed
5. Then the loop variable **i** will be updated. As we have **i++**, value of **i** now will be 2
6. Condition **i<2** is **evaluated** and hold **false** for **i=2**, the body of the for loop will be terminated and statement immediately after loop body will be executed.

Sample for Loop

```
for (i=0 ; i<2 ; i++) {  
    printf("Hello World");  
}  
printf("Outside loop body");
```

OUTPUT

Hello World
Hello World
Outside loop body

Value of i	Condition	State	Execution

Sample for Loop

- What should we change to print hello world 10 times?

```
for (i=0 ; i<10 ; i++) {  
    printf("Hello World\n");  
}
```

for Loop Example 1

- Print all the integer numbers from 1 to 10 inclusive
- What is the lowest number we need to print?
 - 1
- What is the largest number we need to print?
 - 10
- We can run the loop from 1, increase it by value 1 upto 10 and print each value

```
for(i=1; i<=10;i++){  
    printf("%d\n",i);  
}
```

for Loop Example 1

■ Print all the integer numbers from 1 to 10 inclusive

```
for(i=1; i<=10;i++){  
    printf("%d\n",i);  
}
```

Value of i	Condition	State	Execution	Output
i=1	i<=10 or 1<=10	true	Loop body	1
i=2	i<=10 or 2<=10	true	Loop body	2
i=3	i<=10 or 3<=10	true	Loop body	3
i=4	i<=10 or 4<=10	true	Loop body	4
i=5	i<=10 or 5<=10	true	Loop body	5
i=6	i<=10 or 6<=10	true	Loop body	6
i=7	i<=10 or 7<=10	true	Loop body	7
i=8	i<=10 or 8<=10	true	Loop body	8
i=9	i<=10 or 9<=10	true	Loop body	9
i=10	i<=10 or 10<=10	true	Loop body	10
i=11	i<=10 or 11<=10	false	Outside loop	

for Loop Example 2

- **Print the sum of all the integer numbers from 1 to 5 inclusive**

- How many integer numbers are there between 1 to 5 inclusive?

- 5

- How many numbers do we need to add?

- 5 numbers

- What is the lowest number to add in the range?

- 1

- What is the highest number to add in the range?

- 5

- How many times should my loop iterate?

- 5 times

```
int sum=0;
for(i=1; i<=5;i++){
    sum=sum+i;
}
printf("%d",sum);
```


for Loop Example 2

- Print the sum of all the integer numbers from 1 to 5 inclusive

```
int sum=0;
for(i=1; i<=5;i++){
    sum=sum+i;
}
printf("%d",sum);
```

Value of i	Condition	State	Execution	sum=sum+i
i=1	i<5 or 1<=5	true	Loop body	1
i=2	i<5 or 2<=5	true	Loop body	3
i=3	i<5 or 3<=5	true	Loop body	6
i=4	i<5 or 4<=5	true	Loop body	10
i=5	i<5 or 5<=5	true	Loop body	15
i=6	i<5 or 6<=5	false	Outside loop	

for Loop Example 3

▪ Print the following series and its sum:

▪ $1+2+3+4+5=15$

```
int sum=0;
for(i=1; i<=5;i++){
    printf("%d+",i);
    sum=sum+i;
}
printf("\b=%d",sum);
```

Value of i	Condition	State	Execution	sum= sum+i	Output
i=1	$i < 5$ or $1 \leq 5$	true	Loop body	1	1+
i=2	$i < 5$ or $2 \leq 5$	true	Loop body	3	2+
i=3	$i < 5$ or $3 \leq 5$	true	Loop body	6	3+
i=4	$i < 5$ or $4 \leq 5$	true	Loop body	10	4+
i=5	$i < 5$ or $5 \leq 5$	true	Loop body	15	5+
i=6	$i < 5$ or $6 \leq 5$	false	Outside loop		=15

for Loop Example 4

- **Print all the even numbers from 1 to 10 inclusive**
- How many even numbers are there between 1 to 10 inclusive?
 - 5
- How many times do you have to print?
 - 5 times
- How many times should my loop iterate?
 - 5 times
- What is the lowest number to print in the range?
 - 2
- What is the highest number to print in the range?
 - 10

```
for(i=1; i<=10;i++){  
    if(i%2==0){  
        printf("%d\n",i);  
    }  
}
```

for Loop Example 4

■ Print all the even numbers from 1 to 10 inclusive:

```
for(i=1; i<=10;i++){  
    if(i%2==0){  
        printf("%d\n",i);  
    }  
}
```

Value of i	Condition	State	Execution	Output
i=1	i<10 or 1<=10	true	Loop body	
i=2	i<10 or 2<=10	true	Loop body	2
i=3	i<10 or 3<=10	true	Loop body	
i=4	i<10 or 4<=10	true	Loop body	4
i=5	i<10 or 5<=10	true	Loop body	
i=6	i<10 or 6<=10	true	Loop body	6
i=7	i<10 or 7<=10	true	Loop body	
i=8	i<10 or 8<=10	true	Loop body	8
i=9	i<10 or 9<=10	true	Loop body	
i=10	i<10 or 10<=10	true	Loop body	10
i=11	i<10 or 11<=10	false	Outside loop	

for Loop Example 5

■ Print the following series and its sum:

■ $2+4+6+8+10 = 30$

```
int sum=0;
for(i=1; i<=10;i++){
    if(i%2==0){
        printf("%d+",i);
        sum=sum+i;
    }
}
printf("\b=%d",sum);
```

Value of i	Condition	State	Execution	Sum=sum+i	Output
i=1	$i < 10$ or $1 \leq 10$	true	Loop body		
i=2	$i < 10$ or $2 \leq 10$	true	Loop body	2	2+
i=3	$i < 10$ or $3 \leq 10$	true	Loop body		
i=4	$i < 10$ or $4 \leq 10$	true	Loop body	6	4+
i=5	$i < 10$ or $5 \leq 10$	true	Loop body		
i=6	$i < 10$ or $6 \leq 10$	true	Loop body	12	6+
i=7	$i < 10$ or $7 \leq 10$	true	Loop body		
i=8	$i < 10$ or $8 \leq 10$	true	Loop body	20	8+
i=9	$i < 10$ or $9 \leq 10$	true	Loop body		
i=10	$i < 10$ or $10 \leq 10$	true	Loop body	30	10+
i=11	$i < 10$ or $11 \leq 10$	false	Outside loop		

for Loop Example 6(Self Study)

■Print all the odd numbers from 1 to 10 inclusive:

```
for(i=1; i<=10;i++){  
    if(i%2!=0){  
        printf("%d\n",i);  
    }  
}
```

Value of i	Condition	State	Execution	Output
i=1	i<10 or 1<=10	true	Loop body	1
i=2	i<10 or 2<=10	true	Loop body	
i=3	i<10 or 3<=10	true	Loop body	3
i=4	i<10 or 4<=10	true	Loop body	
i=5	i<10 or 5<=10	true	Loop body	5
i=6	i<10 or 6<=10	true	Loop body	
i=7	i<10 or 7<=10	true	Loop body	7
i=8	i<10 or 8<=10	true	Loop body	
i=9	i<10 or 9<=10	true	Loop body	9
i=10	i<10 or 10<=10	true	Loop body	
i=11	i<10 or 11<=10	false	Outside loop	

for Loop Example 7(Self Study)

■ Print the following series and its sum:

■ $1+3+5+7+9 = 25$

```
int sum=0;
for(i=1; i<=10;i++){
    if(i%2!=0){
        printf("%d+",i);
        sum=sum+i;
    }
}
printf("\b=%d",sum);
```

Value of i	Condition	State	Execution	Sum=sum+i	Output
i=1	$i < 10$ or $1 \leq 10$	true	Loop body	1	1+
i=2	$i < 10$ or $2 \leq 10$	true	Loop body		
i=3	$i < 10$ or $3 \leq 10$	true	Loop body	4	3+
i=4	$i < 10$ or $4 \leq 10$	true	Loop body		
i=5	$i < 10$ or $5 \leq 10$	true	Loop body	9	5+
i=6	$i < 10$ or $6 \leq 10$	true	Loop body		
i=7	$i < 10$ or $7 \leq 10$	true	Loop body	16	7+
i=8	$i < 10$ or $8 \leq 10$	true	Loop body		
i=9	$i < 10$ or $9 \leq 10$	true	Loop body	25	9+
i=10	$i < 10$ or $10 \leq 10$	true	Loop body		
i=11	$i < 10$ or $11 \leq 10$	false	Outside loop		

Infinite for loop

- As the name suggests, a loop that iterates for infinite time/ never stops iterating, is called infinite for loop

- **When an infinite loop occurs?**

- Infinite loop occurs if the **condition** of the loop is always true

- **Example:**

```
for(i=1 ; i>0 ; i++){  
    printf("print inside for loop");  
}
```


Jump statements in for loop

- **Break Statement:** When a **break statement** is encountered inside a **loop/switch-case block**, the **loop/switch-case block** is immediately terminated and the program control resumes at the next statement after the **loop/switch-case block**
- **Continue Statement:** When **continue** is encountered inside any loop, the **remaining statement** of the loop body is **skipped** and **control** of the loop is passed to the next step – **update variable**

Jump statements in for loop Example

```
for(i=1; i<=3;i++){  
    printf("%d\n",i);  
    if(i==2){  
        break;  
    }  
    printf("%d\n",i);  
}
```

```
for(i=1; i<=3;i++){  
    printf("%d\n",i);  
    if(i==2){  
        continue;  
    }  
    printf("%d\n",i);  
}
```

Variations of for loop

- Print all the integer numbers from 1 to 3

We can initialize the loop variable before the loop statement.

```
int i=1;  
for(; i<3;i++){  
    printf("%d\n",i);  
}
```

Value of i	Condition	State	Execution	Output
i=1	i<3 or 1<3	true	Loop body	1
i=2	i<3 or 2<3	true	Loop body	2
i=3	i<3 or 3<3	false	Outside loop	

Variations of for loop

■ Print all the integer numbers from 1 to 3

We can update the loop variable inside the loop body.

```
int i=1;
for(; i<3;){
    printf("%d\n",i);
    i++;
}
```

Value of i	Condition	State	Execution	Output
i=1	i<3 or 1<3	true	Loop body	1
i=2	i<3 or 2<3	true	Loop body	2
i=3	i<3 or 3<3	false	Outside loop	

Task – Solve Using for loop

- Print the following series up to n and its sum :
- $1 + 2 + 3 + \dots + n$
- $0 + 2 + 4 + \dots + n$
- $1 + 3 + 5 + \dots + n$
- $1 - 2 + 3 - 4 + \dots n$
- $1^2 + 2^2 + 3^2 + \dots + n^2$
- $x + 2x^2 + 3x^3 + \dots + nx^n$

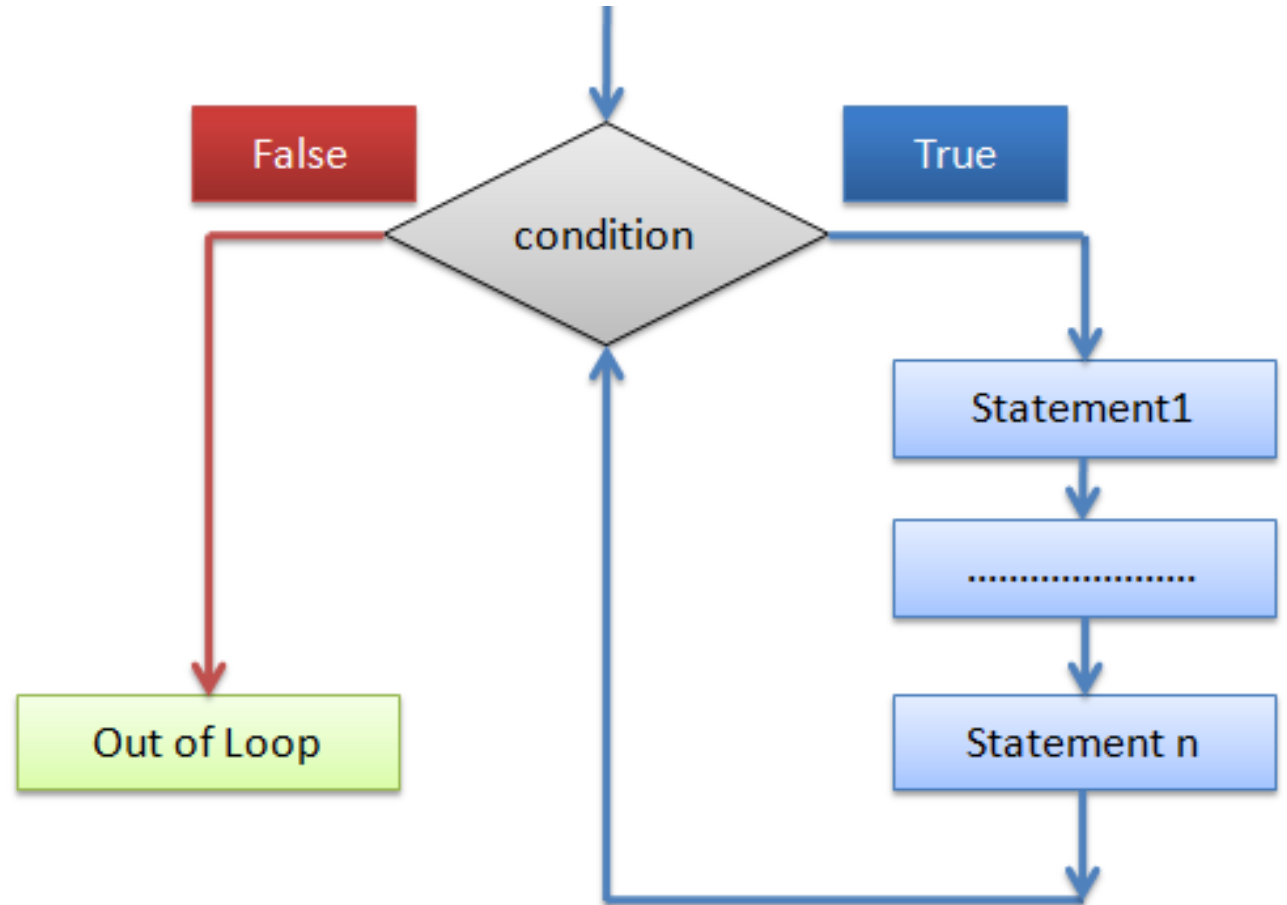
Task - Solve Using for loop

- Print the alphabets from A to Z
- Print the alphabets from a to z
- Print the alphabets from a to z without the vowels
- Take input of a number **n** and print all of its factors
- Take input of two numbers **a** & **b**, and print their GCD(Greatest common divisor)
- Take input of a number and print its factorial

while Loop – Syntax & Flow Chart

■ Syntax:

```
while (conditional test) {  
    statement 1;  
    statement 2;  
    .....  
    statement n;  
}
```



while Loop – Execution Steps

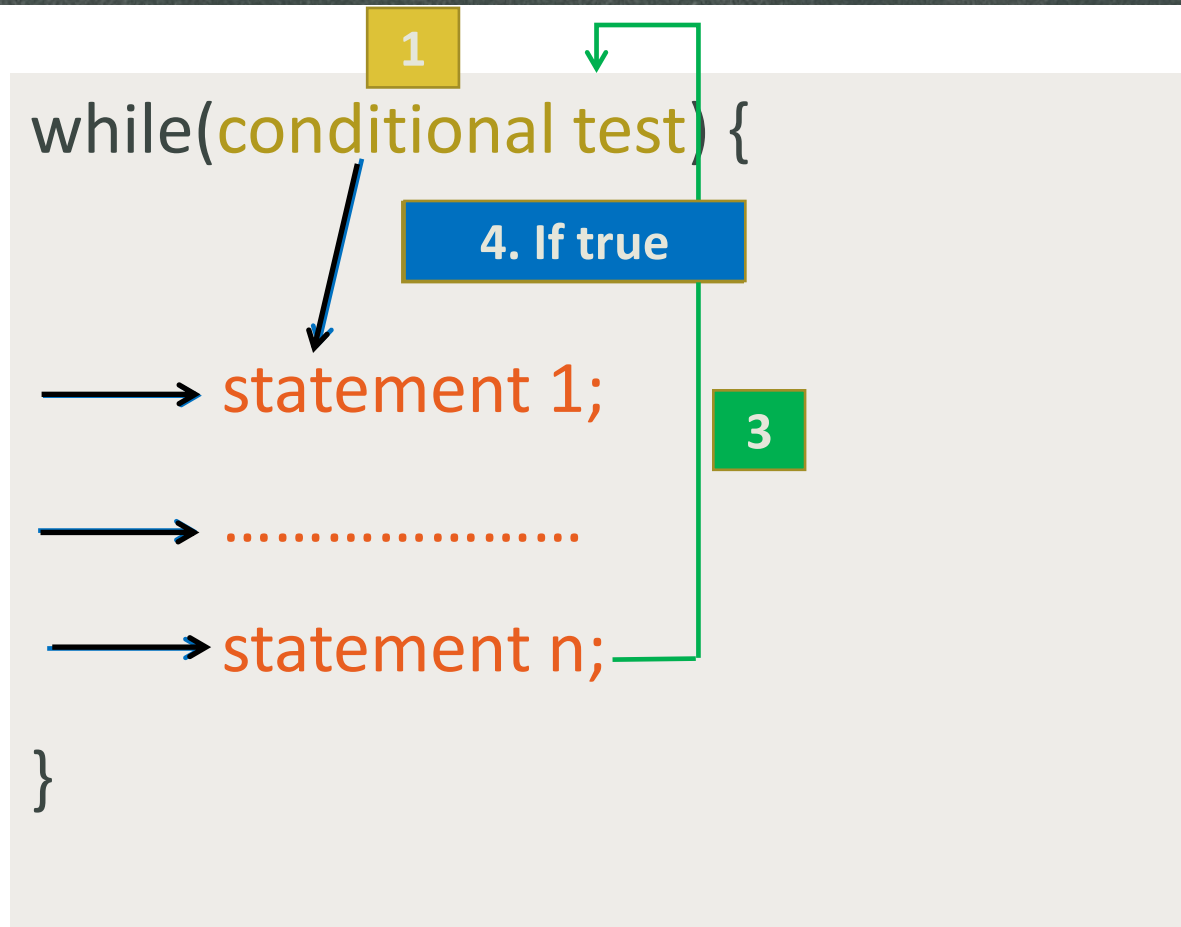
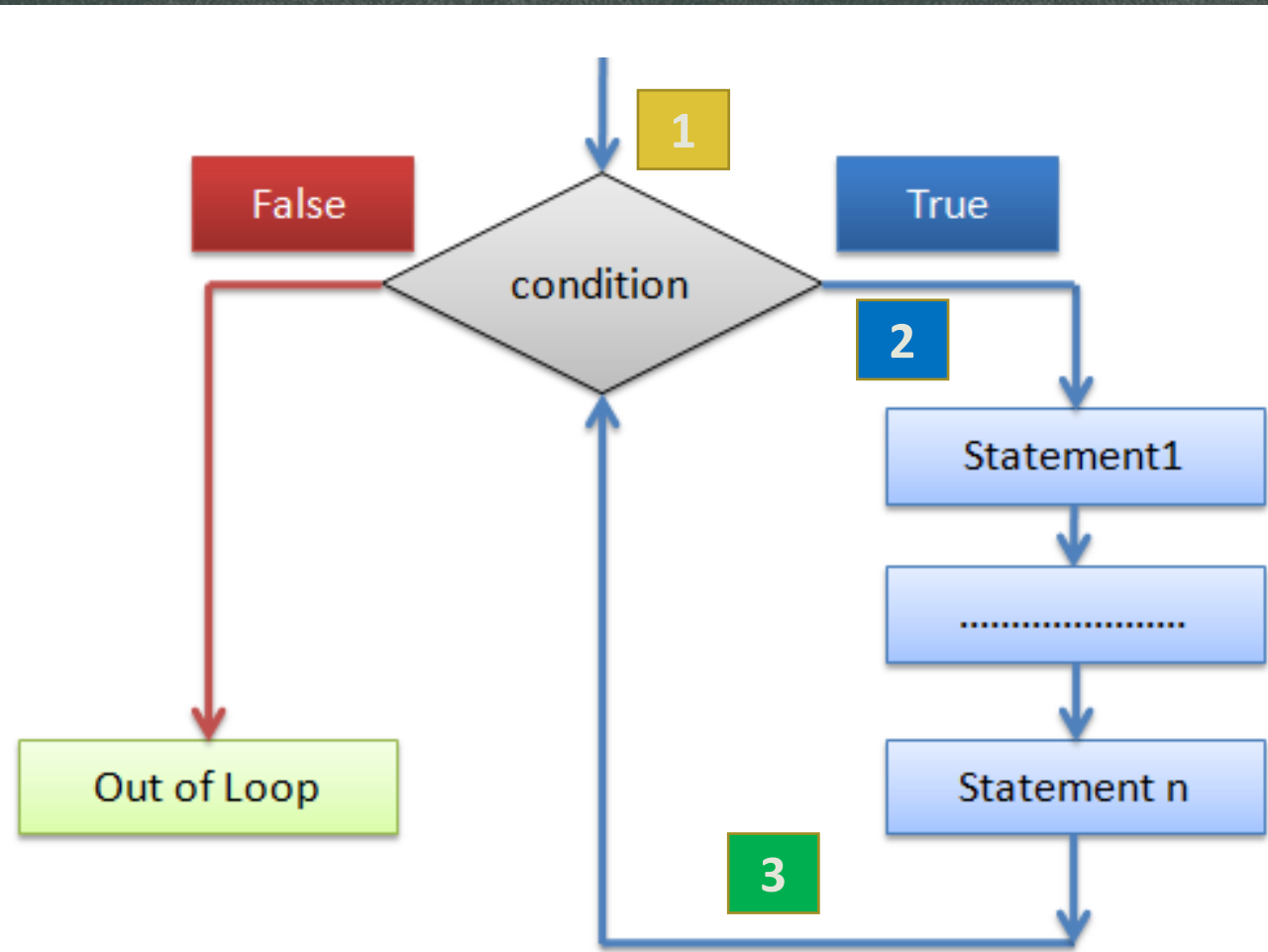
```
while(conditional test) {  
    // body of the loop  
    // statements we want to execute  
}
```

1. Condition is evaluated-

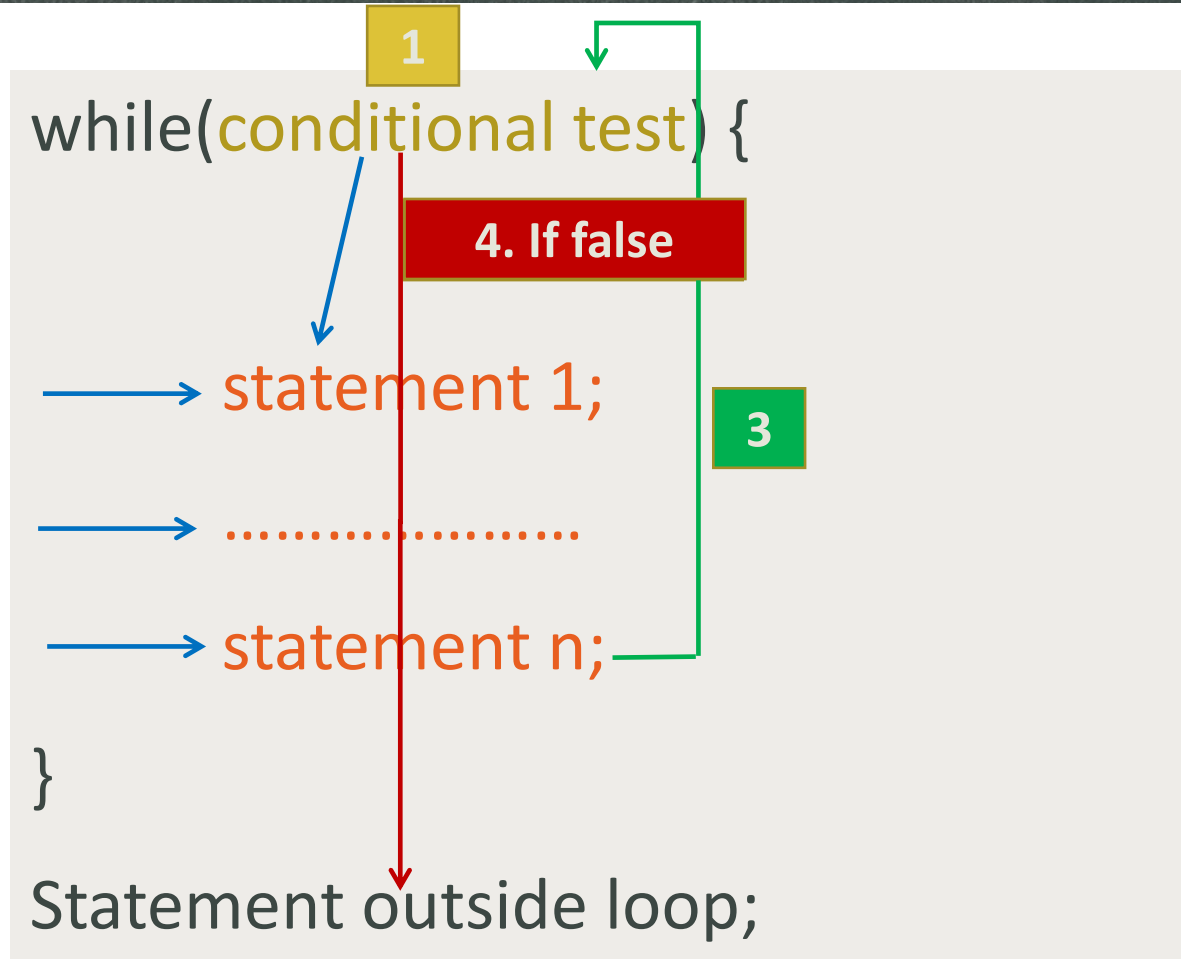
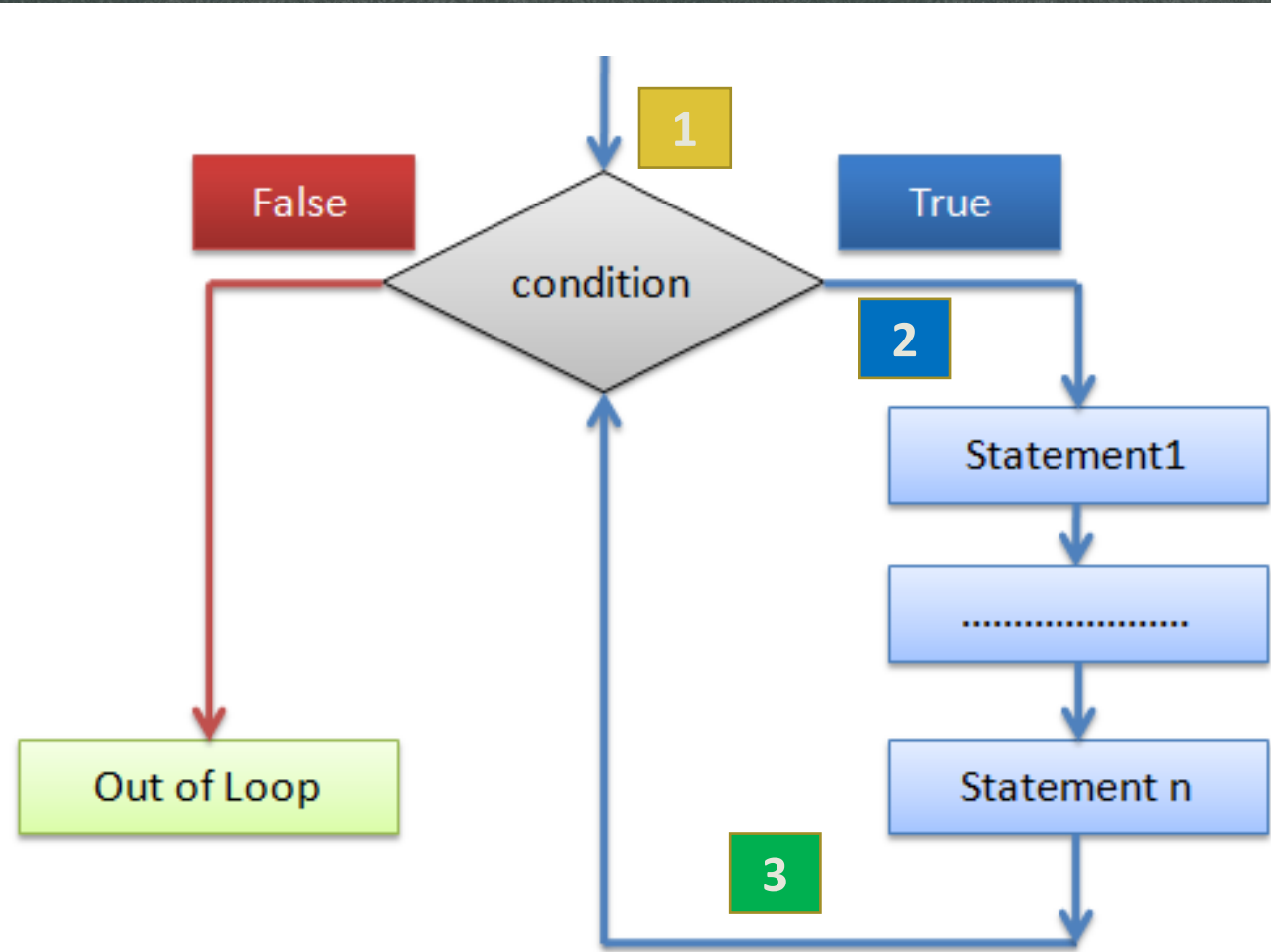
- i. If the condition is **false**, it returns 0 and the **loop** is **terminated**.
- ii. if the condition is **true**, it returns 1, **body of the loop** is executed

2. Step 1 is repeated until the condition is false and loop is terminated

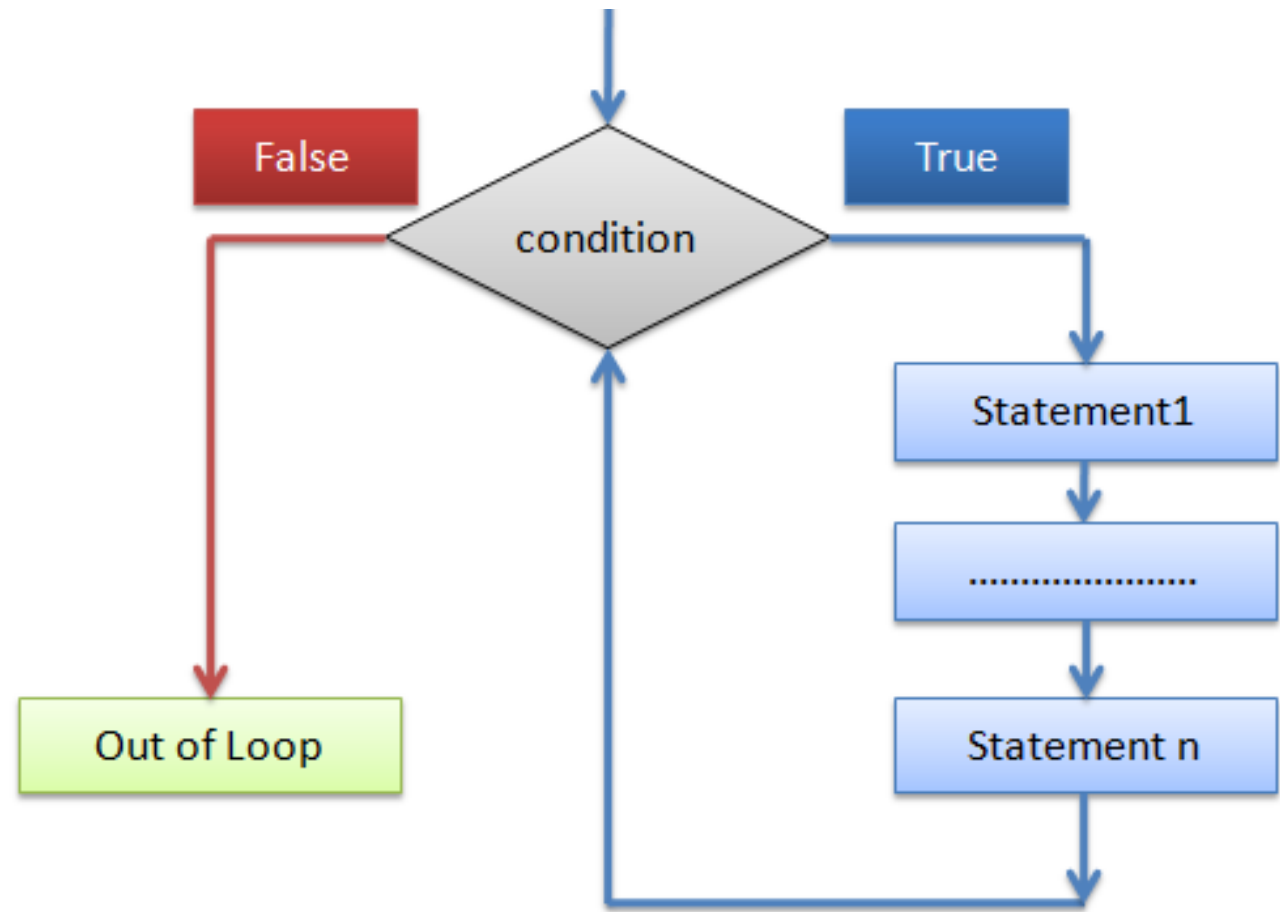
while Loop – Execution Simulation



while Loop – Execution Simulation



Summary of while loop Execution



1. Check condition
2. if **false terminate** loop body
3. If **true execute** loop body
4. repeat from step 1

Sample while Loop

```
1.  int i = 0;
2.  while ( i<2) {
3.      printf("Hello World\n");
4.      i++;
5.  }
6.  printf("Outside loop body");
```

OUTPUT

Hello World
Hello World
Outside loop body

1. Here at line 1 **i** is **initialized** with **0**
2. Condition **i<2** is **evaluated**. Since now value of **i** is **0** which makes the condition **i<2** **true**, the body of the while loop will be executed.
3. At line 4, as we have **i++**, value of **i** now will be 1
4. Condition **i<2** is **evaluated** and hold **true** for **i=1**, the body of the while loop will be executed
5. Again at line 4, we have **i++**, value of **i** now will be 2
6. Condition **i<2** is **evaluated** and hold **false** for **i=2**, the body of the while loop will be terminated and statement immediate after loop body will be executed.

Sample while Loop

```
1. int i = 0;  
2. while ( i<2) {  
3.     printf("Hello World");  
4.     i++;  
5. }  
6. printf("Outside loop body");
```

OUTPUT

Hello World

Hello World

Outside loop body

Value of i	Condition	State	Execution

while Loop Example 1

- Print all the integer numbers from 1 to 10 inclusive

```
for(i=1; i<=10;i++){  
    printf("%d\n",i);  
}
```

```
int i=1;  
while(i<=10){  
    printf("%d\n",i);  
    i++;  
}
```

while Loop Example 2

- Print all the odd numbers from 1 to 10 inclusive

```
for(i=1; i<=10;i++){  
    if(i%2!=0){  
        printf("%d\n",i);  
    }  
}
```

```
int i=1;  
while(i<=10){  
    if(i%2!=0){  
        printf("%d\n",i);  
    }  
    i++;  
}
```

Task – Solve Using while loop

Print the following series up to n and its sum :

$1 + 2 + 3 + \dots + n$

$0 + 2 + 4 + \dots + n$

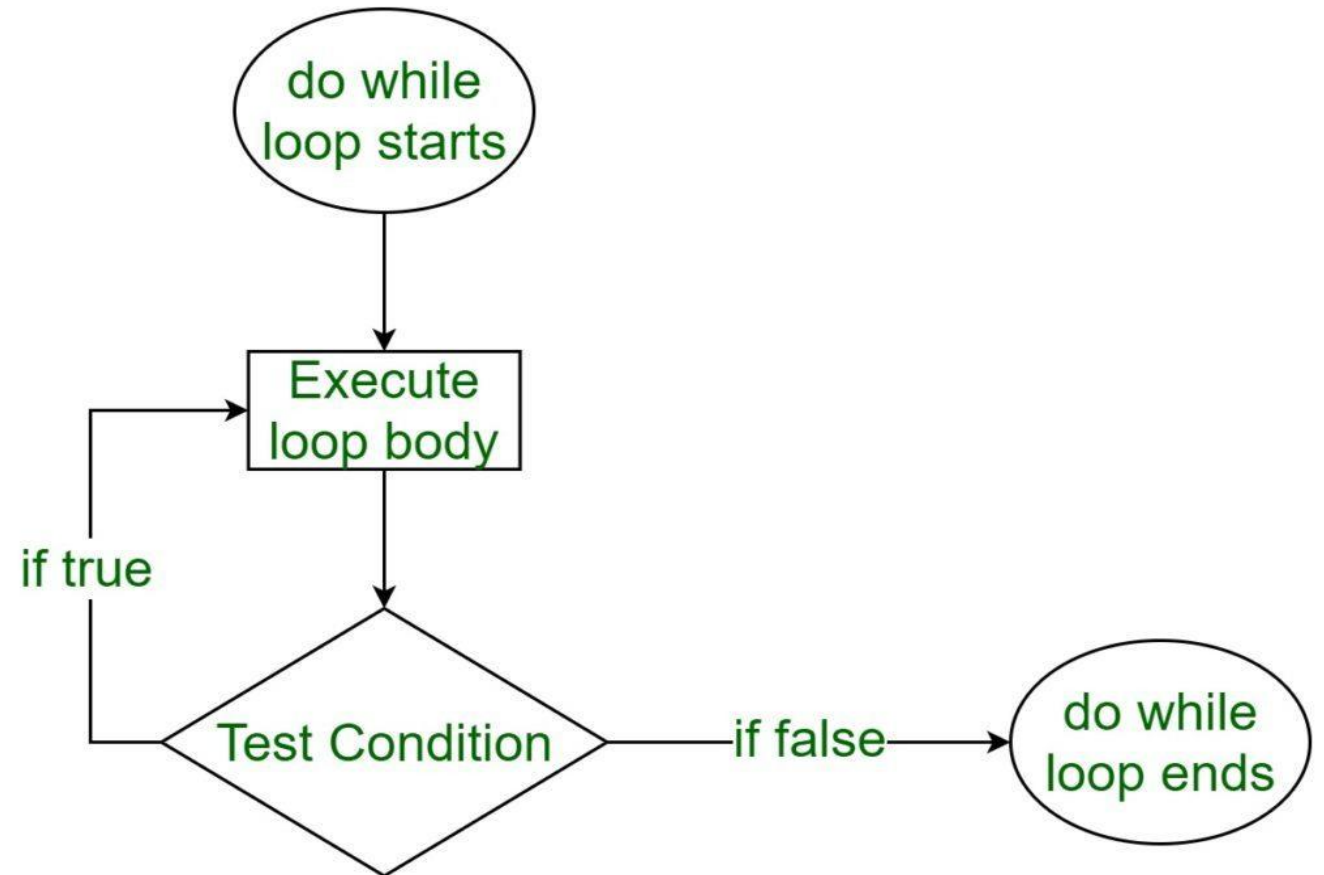
$1 + 3 + 5 + \dots + n$

- Print all the even numbers from 1 to 10 inclusive
- Print the reverse of a given number
- Take input of a number **n** and print all of its factors

do while Loop – Syntax & Flow Chart

■ Syntax:

```
do{  
    statement 1;  
    statement 2;  
    .....  
    statement n;  
} while (conditional test) ;
```

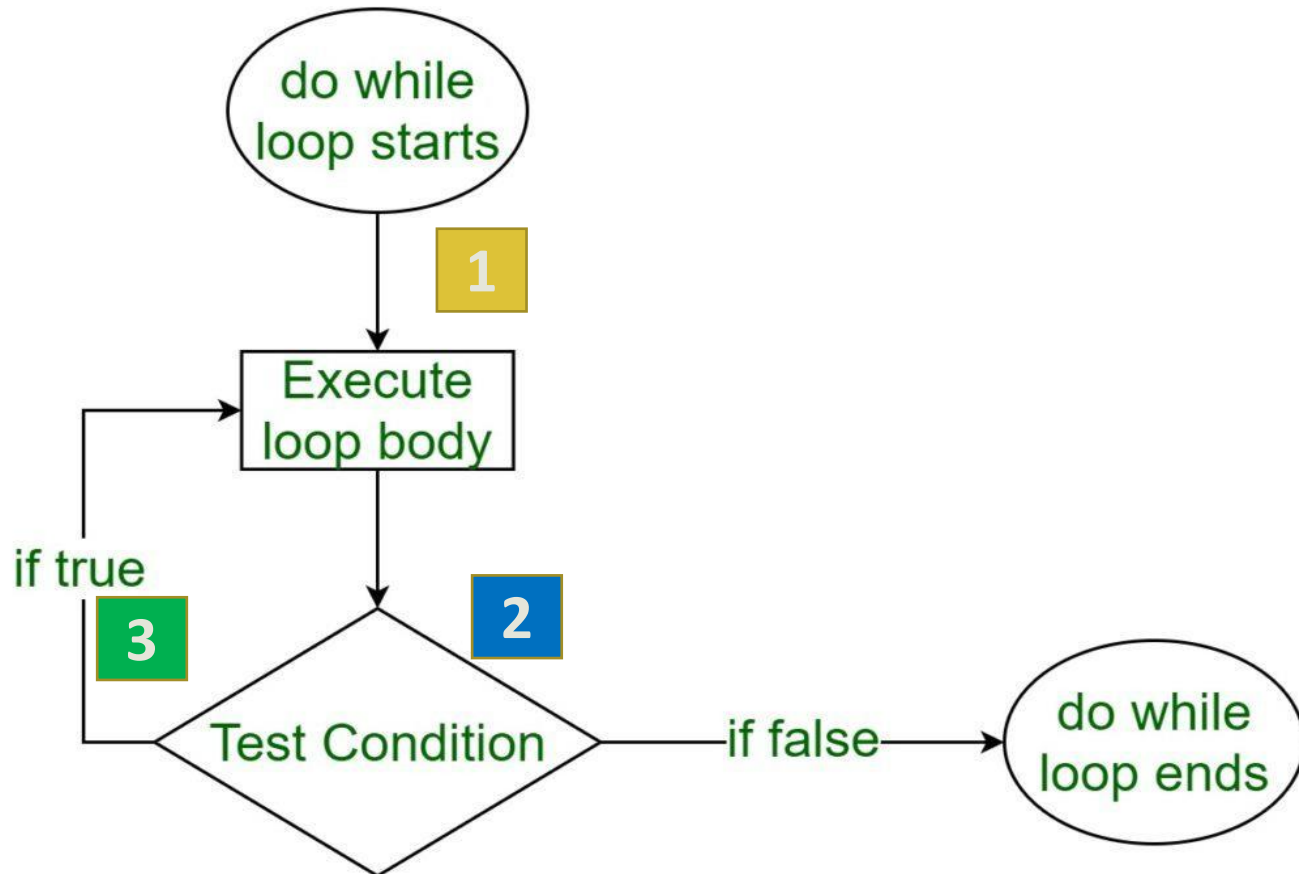


do while Loop – Execution Steps

```
do{  
    // body of the loop  
    // statements we want to execute  
} while(conditional test) ;
```

1. Body of the loop is executed first.
2. **Condition** is **evaluated**-
 - i. If the condition is **false**, it returns 0 and the **loop** is **terminated**.
 - ii. if the condition is **true**, it returns 1, **body of the loop** is executed
2. **Step 2 is repeated until the condition is false and loop is terminated**

do while Loop – Execution Simulation



do{ 1

→ statement 1;

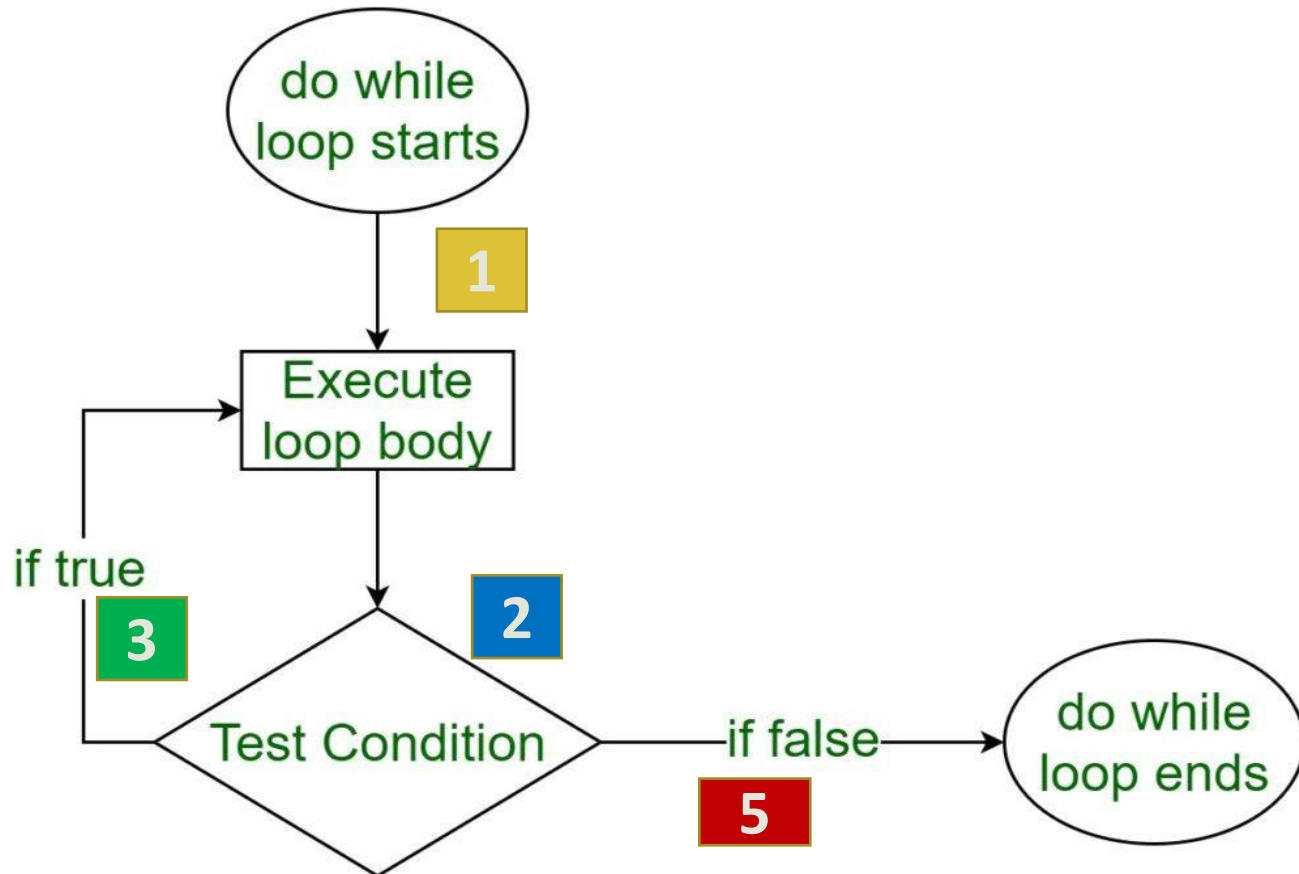
→

→ statement n;

} while(conditional test) ; 4

5. If true

do while Loop – Execution Simulation



do{ 1

→ statement 1;

→

→ statement n;

} while(conditional test); 4

Statement outside loop;

3. If true

5. If false

Sample do while Loop

```
1. int i = 0;  
2. do{  
3.     printf("Hello World");  
4.     i++;  
5. } while ( i<2) ;  
6. printf("Outside loop body");
```

OUTPUT

Hello World

Hello World

Outside loop body

Execution	Value of i	Condition	State
Loop body	i=1	i<2 or 0<2	true
Loop body	i=2	i<2 or 1<2	false
Outside loop			

Difference Between while and do while Loop

- In while loop, first the condition is checked, then the body is executed if condition is true.
- In do...while loop, first the body is executed, then condition is checked.
- the body of do...while loop will execute at least once irrespective to the test-expression.

Difference Between while and do while Loop

```
int i=0;
while(i<0){
    printf("I is a positive number");
    i++;
}
```

```
int i=0;
do{
    printf("I is a negative number");
    i++;
}while(i<0);
```

Difference Between while and do while Loop

```
int i=0;
while(i<0){
    printf("I is a positive number");
    i++;
}
```

Value of i	Condition	State	Execution
i=0	i<0 or 0<0	false	Outside loop

Difference Between while and do while Loop

```
int i=0;
do{
printf("I is a negative number");
    i++;
}while(i<0);
```

Output:

I is a negative number

Execution	Value of i	Condition	State
Loop body	i=1	i<0 or 1<0	false
Outside loop			

*Thank
you!*